

Основы Nuxt.js

Что такое Nuxt.js?

Nuxt.js — это фреймворк, построенный на основе Vue.js, который упрощает разработку универсальных и статических приложений. Он предоставляет архитектуру и множество функций для создания современных веб-приложений.

Ключевые особенности

- **Серверный рендеринг (SSR):** Улучшает SEO и производительность
- **Статическая генерация (SSG):** Создает статические файлы для быстрой загрузки
- **Автоматическая маршрутизация:** Создает маршруты на основе структуры файлов
- **Разделение кода:** Автоматически разделяет код для оптимальной загрузки
- **Горячая замена модулей (HMR):** Обновляет приложение без перезагрузки страницы
- **Экосистема модулей:** Расширяет функциональность с помощью модулей

Установка и создание проекта

```
# Установка с помощью npx
npx nuxi init my-nuxt-app

# Переход в директорию проекта
cd my-nuxt-app

# Установка зависимостей
npm install

# Запуск сервера разработки
npm run dev
```

Структура проекта

├ .nuxt/	# Автоматически генерируемая директория (не изменять)
├ assets/	# Необработанные ресурсы (CSS, изображения и т.д.)
├ components/	# Vue компоненты
├ composables/	# Композабельные функции
├ content/	# Контент (с модулем @nuxt/content)
├ layouts/	# Макеты приложения
├ middleware/	# Функции промежуточного ПО
├ pages/	# Страницы приложения (автоматическая маршрутизация)
├ plugins/	# Плагины Vue.js
├ public/	# Статические файлы (доступны по корневому URL)
├ server/	# Серверные маршруты API и промежуточное ПО
└ app.vue	# Главный компонент приложения

```
└─ nuxt.config.ts    # Конфигурация Nuxt
└─ package.json      # Зависимости и скрипты проекта
```

Жизненный цикл Nuxt.js

1. **Инициализация Nuxt:** Загрузка nuxt.config.js и плагинов
2. **Инициализация Vue:** Создание экземпляра Vue
3. **Серверный рендеринг** (если включен SSR)
4. **Гидратация** (на клиенте)
5. **Клиентская навигация**

Режимы рендеринга

Универсальный режим (SSR)

```
// nuxt.config.js
export default {
  ssr: true
}
```

Статический режим (SSG)

```
// nuxt.config.js
export default {
  ssr: true,
  target: 'static'
}
```

Клиентский режим (SPA)

```
// nuxt.config.js
export default {
  ssr: false
}
```

Примеры использования

Базовая страница

```
<!-- pages/index.vue -->
<template>
  <div>
```

```
<h1>Главная страница</h1>
<p>Добро пожаловать в мое Nuxt.js приложение!</p>
</div>
</template>
```

Использование асинхронных данных

```
<!-- pages/users.vue -->
<script setup>
const { data: users } = await useFetch('/api/users')
</script>

<template>
  <div>
    <h1>Пользователи</h1>
    <ul>
      <li v-for="user in users" :key="user.id">{{ user.name }}</li>
    </ul>
  </div>
</template>
```

Что нужно знать про Nuxt на собеседовании

Отлично, давайте разберем ключевые аспекты Nuxt, о которых вас могут спросить на собеседовании. Подготовьтесь к тому, что вопросы будут варьироваться в зависимости от уровня позиции (junior, middle, senior), но следующие темы являются базовыми и важными.

Основные концепции и принципы Nuxt:

- **Что такое Nuxt.js?**
 - Это фреймворк для создания универсальных (SSR/SPA), статических веб-приложений и API на основе Vue.js.
 - Он предоставляет готовую структуру и множество встроенных возможностей, упрощая и ускоряя разработку.
 - Подчеркните его философию "Convention over Configuration" (соглашения вместо конфигурации).
- **Основные преимущества использования Nuxt:**
 - **Server-Side Rendering (SSR):** Улучшает SEO, ускоряет первоначальную загрузку страницы (Time To First Contentful Paint - TTFCP), обеспечивает лучший UX для пользователей с медленным интернетом или устаревшими устройствами.
 - **Static Site Generation (SSG):** Позволяет генерировать полностью статические сайты, что обеспечивает высокую производительность и безопасность.
 - **Автоматическая генерация роутов:** На основе структуры каталога [pages](#).

- **Модульная архитектура:** Легко расширять функциональность с помощью готовых или собственных модулей.
- **Структурированная организация проекта:** Предлагает четкую структуру каталогов (`pages`, `components`, `layouts`, `store`, `plugins`, `middleware`, `assets`, `static`).
- **Удобная работа с мета-тегами:** Компонент `<NuxtHead>`.
- **Встроенная поддержка Vuex (через `@nuxtjs/store`):** Для управления состоянием приложения.
- **Простота работы с API (через `@nuxtjs/axios`):** Для выполнения HTTP-запросов.
- **Различные режимы развертывания:** `ssr`, `static`, `spa`.
- **Основные отличия Nuxt от чистого Vue.js:**
 - Nuxt предоставляет готовую архитектуру и множество встроенных решений "из коробки", в то время как Vue.js - это более гибкая библиотека, требующая самостоятельной настройки многих аспектов (роутинг, SSR и т.д.).
 - Nuxt ориентирован на создание более крупных и сложных приложений с акцентом на SSR и SSG.

Структура проекта Nuxt:

- **`pages/`:** Содержит Vue-компоненты, которые автоматически становятся страницами приложения на основе структуры каталогов.
- **`components/`:** Переиспользуемые Vue-компоненты.
- **`layouts/`:** Определяет макеты страниц (например, основной макет, макет для авторизованных пользователей). Файл `default.vue` является стандартным макетом.
- **`store/`:** Содержит файлы Vuex для управления состоянием приложения (если используется `@nuxtjs/store`). Может быть как в "классическом" режиме (`index.js`, `mutations.js`, `actions.js`), так и в "модульном" режиме (подкаталоги с файлами состояния, мутаций, действий).
- **`plugins/`:** Используется для регистрации Vue-плагинов, которые необходимо выполнить перед созданием корневого экземпляра Vue.
- **`middleware/`:** Содержит функции-перехватчики, которые могут выполняться до или после навигации по маршрутам (глобальные, на уровне макета, на уровне страницы).
- **`assets/`:** Содержит некомпилируемые статические ресурсы (изображения, шрифты и т.д.).
- **`static/`:** Содержит статические файлы, которые напрямую копируются в корневой каталог сборки (`robots.txt`, `favicon.ico` и т.д.).
- **`nuxt.config.js`:** Основной файл конфигурации Nuxt, где определяются различные настройки приложения (модули, CSS, сборка, маршрутизация и т.д.).

Ключевые файлы и конфигурации:

- **`nuxt.config.js`:** Будьте готовы объяснить основные опции, такие как `modules`, `css`, `build`, `head`, `router`, `server`, `env`.
- **Роутинг:** Как Nuxt автоматически генерирует роуты на основе каталога `pages`. Понимание динамических маршрутов (с префиксом `_`).
- **Макеты (`layouts/`):** Как создавать и использовать различные макеты.
- **Компоненты:** Разница между компонентами в `components/` и страницами в `pages/`.
- **Хранилище (`store/`):**

- Принцип работы Vuex.
- Разница между Mutations, Actions и Getters.
- Как Nuxt упрощает работу с Vuex (если используется `@nuxtjs/store`).
- Понимание "классического" и "модульного" режимов.
- **Плагины (`plugins/`):** Как регистрировать и использовать плагины. Укажите на жизненный цикл плагинов (client-side only, server-side only).
- **Middleware (`middleware/`):**
 - Различные типы middleware (глобальные, макета, страницы).
 - Порядок их выполнения.
 - Примеры использования (авторизация, логирование).
- **Активы и статика:** Разница между каталогами `assets/` и `static/`.

Работа с данными:

- **`asyncData`:**
 - Основное назначение: получение данных на сервере (для SSR) или на клиенте (для SPA) *перед* отрисовкой компонента страницы.
 - Контекст, который передается в `asyncData` (params, query, store, \$axios и т.д.).
 - Возвращаемое значение `asyncData` (объект, который сливается с данными компонента).
 - Отличия от `mounted` (выполняется только на клиенте).
- **`fetch`:**
 - Альтернативный хук для получения данных, появившийся в Nuxt 2.12+.
 - Более гибкий и предоставляет больше контроля над процессом получения данных (например, можно отменить запрос).
 - Доступ к контексту через `this.$fetchState`.
 - Возможность отслеживать состояние загрузки.
- **Работа с API:**
 - Использование `@nuxtjs/axios`.
 - Конфигурация Axios в `nuxt.config.js`.
 - Обработка ошибок.

Развертывание (Deployment):

- Основные режимы развертывания Nuxt: `ssr`, `static`, `spa`.
- Понимание различий между ними и когда какой режим следует использовать.
- Общие стратегии развертывания (например, на Node.js сервере, на статических хостингах).

Продвинутые темы (в зависимости от уровня позиции):

- **Модули Nuxt:**
 - Как работают модули.
 - Примеры популярных модулей (`@nuxtjs/axios`, `@nuxtjs/auth-next`, `@nuxtjs/i18n`).
 - Как создавать собственные модули (основные принципы).
- **Хуки Nuxt:** Понимание жизненного цикла Nuxt и доступных хуков.
- **Производительность:** Оптимизация Nuxt-приложений (code splitting, lazy loading, кеширование).

- **Тестирование:** Как тестировать Nuxt-приложения (unit, integration, e2e).
- **TypeScript:** Интеграция TypeScript с Nuxt ([@nuxt/typescript-build](https://nuxt.com/docs/guide/typescript-build)).
- **Композиционный API (Composition API) в Nuxt:** Как использовать Composition API внутри Nuxt-приложений.
- **Nuxt 3 (если актуально для позиции):** Основные отличия от Nuxt 2 (Vite, Composition API по умолчанию, Nitro).

Советы для собеседования:

- **Будьте готовы объяснить "почему":** Не просто называйте функции Nuxt, но и объясняйте, зачем они нужны и какие проблемы решают.
- **Приводите примеры из своего опыта:** Если у вас есть опыт работы с Nuxt, обязательно упоминайте конкретные проекты и задачи, которые вы решали.
- **Покажите понимание концепций:** Важно не просто знать синтаксис, но и понимать основные принципы SSR, SSG, управления состоянием и т.д.
- **Не бойтесь признаться, что чего-то не знаете:** Лучше честно сказать, что вы не сталкивались с определенной темой, чем пытаться дать неверный ответ.
- **Задавайте вопросы:** Покажите свою заинтересованность, задавая продуманные вопросы о технологическом стеке компании, процессах разработки и ожиданиях от кандидата.

Удачи на собеседовании! Тщательная подготовка по этим темам значительно повысит ваши шансы на успех.

Заключение

Nuxt.js предоставляет мощный и гибкий фреймворк для создания современных веб-приложений с Vue.js. Он значительно упрощает разработку, предоставляя готовые решения для маршрутизации, управления состоянием, серверного рендеринга и многого другого.